CHAPTER 1: WHAT YOU ARE ACTUALLY BEING ASKED TO DECIDE

Summary: *Somebody hands you a business problem and expects an architecture back. The gap between those two things is the whole job, and it is where most of Solution Design lives. This chapter closes the gap with one move: stop asking what Claude should do, and start asking what work is being delegated, to whom, and what it costs when the delegate is wrong. Then it walks the four stages of an end-to-end design and shows which one gets dropped, why it gets dropped, and what the omission destroys.*

An Accuracy Number Is Not a Design Input

Suppose the volume is 14,000 claims a month and a pilot comes back 99.7% accurate. That figure will be presented to you as a result. It is not a **result**. It is an **input to a calculation nobody has done** yet, and doing it takes ten seconds.

99.7% of 14,000 leaves **41 claims a month** routed **somewhere they should not have gone**, which is about two every working day. The percentage describes how the system behaves. The count describes what the business has to absorb, and only the second one is a thing you can design against.

Then ask which **41**. That question is the whole diagnostic move in this chapter, and the reason it works is that a rate carries no information about which specific instances fall on the wrong side of it.

Forty-one duplicate address updates is a rounding error and an afternoon of cleanup. Forty-one denied medical claims is people who did not get treatment, a regulator who would like a word, and eventually a headline. Same accuracy, same system, two architectures with almost nothing in common: the first needs a cleanup job, the second needs a person in the path before anything reaches a claimant.

No accuracy figure can tell you which of those you are **building**. That is a permanent property of rates, not a reporting gap somebody can close by measuring more carefully.

Four Model Properties Fixed Before You Design Anything

Four properties of the technology are settled before anyone draws anything, and every one of them constrains the design whether or not the design accounts for it.

The **model is not deterministic**. Identical inputs can produce different outputs even with temperature pinned to 0, on Anthropic's own inference service and on third-party providers.[1] That sentence is worth a second pass for anyone who has ever promised a stakeholder that a given input will always yield a given result. A requirement phrased as "the same case must always get the same answer" is not a requirement the model satisfies on its own. It is a requirement satisfied by a deterministic component sitting beside the model, and that component has to appear in the architecture.

The **context window is working memory, not storage**. It is finite, and accuracy and recall degrade as it fills, a phenomenon Anthropic names context rot.[2] A design that simply keeps appending has a cliff in it somewhere, and production is an expensive place to discover where.

The **model does not know the business**. Anthropic publishes a reliable knowledge cutoff, the date through which a model's knowledge is most extensive and reliable, separately from the broader training data cutoff.[3] Neither date covers the pricing table, the policy exceptions, or the thing the operations lead changed last Tuesday. Facts owned by the company have to be handed to the model, every time, by something.

Wrongness is detectable, but only if **somebody builds the detector**. Anthropic documents concrete verification techniques: chain-of-thought verification, running the same prompt repeatedly and comparing the outputs, iterative refinement, and constraining the model to supplied documents.[4] None of them happen automatically. Each is a component that has to be designed, paid for in latency, and maintained.

Hold those four beside the 41 before you draw a single box. The reason one accuracy percentage could not tell anyone what to build is that it never contained one word of this.

Three Owners of Work, and How Each One Fails

"What should Claude do here?" is the wrong opening question, because it assumes the answer is a quantity of Claude and asks you to pick one. Replace it with a question that produces a design instead of a size: for every piece of work in this process, **who owns** it?

There are exactly three candidates. Assigning work to the wrong one is the most expensive mistake in this domain, and it is expensive specifically because the resulting system still works most of the time.

Owner	Give it	Never give it	How it fails
Claude	Judgement over unstructured input. Language, ambiguity, anything where a competent person would say "it depends."	Arithmetic that has to reconcile. Anything requiring an identical answer every time. Any fact that lives in a system of record.	Plausibly. It returns a well-formed answer that happens to be wrong, and nothing in the output looks different from a right one.
Existing deterministic systems	Calculation, lookup, entitlement, validation, and the authoritative value of every owned fact.	Interpretation of anything a person wrote in free text.	Loudly. It throws, times out, or returns an error code. Somebody finds out.

Owner	Give it	Never give it	How it fails
Humans	Decisions somebody has to be accountable for. Exceptions. The cases the design did not anticipate.	Volume. Anything routed to them because the design declined to decide.	Slowly, then all at once. Attention degrades long before throughput does.

That third column is not decoration. Tools are the documented mechanism for connecting Claude to external systems and APIs,[5] and what a tool actually buys is a change of owner: after the call returns, the external system is the authority on that fact and the model is not. So when a design asks the model to recall a customer's credit limit, the defect is not prompt quality and no amount of prompt engineering will repair it. The defect is that a fact is held by the wrong owner.

The failure column matters more than either of the middle two. Deterministic systems fail loudly and people fail slowly, and existing monitoring was built for exactly those two shapes. Claude fails plausibly, which none of that tooling understands, because a confident wrong answer emits no error, trips no threshold, and looks in every log exactly like a confident right one. That asymmetry is the reason the architecture around the model is always larger than the model.

Placing the Delegation Boundary: Three Questions

The line separating work handed to the model from work kept under deterministic or human control is the **delegation boundary**. Drawing it is not a step in the design. It is the design, and the rest of the architecture is bookkeeping around it.

Almost everything else in Solution Design is a consequence of where it sits. Pattern selection asks how much work sits on the model's side of the line. Multi-agent orchestration asks whether that side needs subdividing. Decomposition is the technique for moving the line after the first placement turned out too generous.

Three questions place it, in this order.

1. What does this step **cost when it is wrong** and **nobody notices for a week**? Not how often it is wrong. Wrongness is a given, and the useful question is what the wrongness accumulates while it is invisible.

2. Can the **wrong outcome be undone**? A draft can be deleted. A payment can be clawed back with effort and an apology. A message sent to a customer cannot be unsent, and a deleted record cannot be undeleted.

3. Whose **name is on it when it is wrong**? When the answer is a named individual with a professional obligation, that person needs enough information to genuinely own the decision, and supplying it is a constraint on your design rather than a training problem.

One seductive wrong answer deserves naming here, because it turns up constantly and it sounds like diligence. The **confidence-threshold trap**: route to a human whenever the model reports low confidence, and the risk is covered. It fails on precisely the cases worth covering. Those 41 misrouted claims were misrouted while the model was confident, and a design that escalates only on low confidence escalates on everything except the failures that hurt.

Stakes route work. Confidence does not.

Four Named Competencies, and Where Each One Lives in This Book

The delegation boundary has a name outside this book, and so do three of its neighbours. Anthropic's **AI Fluency** framework sets out **four competencies**, known as the **4Ds**, and a scenario will sometimes name one of them and expect you to know which kind of answer it is asking for.

Competency	What it decides
Delegation	What work to do **yourself**, what to do with the **model**, what to let it **handle alone**

Competency	What it decides
Description	How to **tell the system what you want**, clearly enough that it can do the work
Discernment	How to **judge what came back**, how it was **produced**, and how the system **behaved**
Diligence	Who takes **responsibility** for the collaboration and for what **gets shipped** from it

Each has three named parts, **twelve in all**, and there is no thirteenth.

Delegation is **Problem Awareness**, understanding the goal and the work; **Platform Awareness**, knowing what the system can and cannot do; and **Task Delegation**, dividing the work between human and model. That is this chapter, with the boundary question standing in for all three.

Description is **Product Description**, what you want made; **Process Description**, how you want it approached; and **Performance Description**, how you want the system to behave while doing it. That is chapter 8.

Discernment is **Product Discernment**, whether the output was any good; **Process Discernment**, whether the route to it was sound; and **Performance Discernment**, whether the interaction itself worked. That is chapters 13 and 17.

Diligence is **Creation Diligence**, which system and used how; **Transparency Diligence**, who gets told what the model did; and **Deployment Diligence**, **verifying and vouching** for what you ship. That is chapters 15 and 16.

Two structural facts are worth more than the list.

Description and Discernment are **one loop seen from two ends**. Description is you **telling the system what you want**; Discernment is you **judging what came back**. They run as a **cycle** rather than as two separate skills, and that has a consequence you will use: a **Discernment finding usually implies a Description repair**. When a

scenario reports that a system produced something unsatisfactory, the fix often sits on the **Description side** even though the observation is a **Discernment one**.

The four also apply across **three ways of working**. In **automation** the system **executes your instructions**. In **augmentation** you and it **work as partners**. In **agency** it **works independently on your behalf**, and you shape its **knowledge and behaviour** rather than its individual actions. Nearly every scenario on this exam is the third. That is why **Delegation and Diligence carry the weight** here: once the system acts on its own, what it may **act** on and who **answers for the result** is the whole job.

One rule falls out of the vocabulary and settles several scenarios on its own.

A **Discernment observation** states something about the **output**, the **reasoning that produced it**, or the **system's behaviour**. It is never a complaint about how a **study was run**.

Sample size, response rates, pilot duration, whether the right people were surveyed: those are **research-design caveats**. They hold for nearly any pilot, they can be raised **without knowing anything about the system** in front of you, and raising one is **not discernment**. It is **changing the subject**. Two questions separate them. Could this have been **written before seeing any result**? If yes, it is a caveat. Does it name **something the system produced**, or **something it failed to consider**? If yes, it is **Discernment**.

The Four Stages, and the One That Gets Dropped

The blueprint asks for end-to-end architecture and it names the stages: input, processing, output, feedback loops. Four stages. Most designs presented for review have three.

Stage	The decision	The question that settles it	Skip it and

Stage	The decision	The question that settles it	Skip it and
Input	Where does work enter, and in what shape?	What is the trigger, and does it arrive once, at least once, or unreliably?	Volume, batching and latency assumptions get made by accident instead of on purpose.
Processing	Who owns each step, on which side of the boundary?	For every fact used, is the model recalling it or being handed it?	The model quietly becomes the authority on things it cannot know.
Output	Where does the result land, and what acts on it?	Does anything downstream treat this as authoritative, and is it entitled to?	A draft becomes a decision, because nothing in the pipe distinguishes them.
Feedback	How does anyone learn this was wrong?	What signal reaches a named person, how fast, and what are they expected to do?	The result is a system with no way to report that it is failing.

The fourth row is the one that gets dropped, and it is dropped for a reason that sounds entirely sensible in the room. Feedback is post-launch. It is a phase-two concern. The pilot does not need it.

It does. A feedback loop is not a reporting feature bolted on after go-live, it is the only mechanism by which anybody ever finds out about the 41, and it has to exist at design time because it constrains all three of the other stages. It decides what gets logged during processing, which identifier has to survive as far as the output, and

how long any of it is retained. Retrofitting those three later means changing the parts of the system that were already signed off.

Watch for a second trap in the same area. The **dashboard failure mode**: the design includes observability, so feedback is handled. A dashboard is a place where a signal waits for somebody to look at it. Unless something routes a signal to a specific person on a schedule, the loop is open, and an open loop with excellent charts is still an open loop.

Design the loop before you build the demo. The demo will never ask for it, which is the whole problem.

How Wrong Answers Are Built on This Material

Wrong answers on this material are not stupid. Almost every one of them describes something a competent architect would actually do, which is why it survives a first read and why elimination by obvious-nonsense does not work here. What makes it wrong is a single assumption underneath it, and there are only four assumptions worth memorising.

- **Improvement mistaken for guarantee**. The option does something that raises the chance of a correct result and is offered as making the result certain. Rewording instructions, showing reasoning, splitting a task into stages: all real, none exact. Whenever a scenario needs an outcome to be guaranteed, the answer moves the decision to something that computes.

- **Detection mistaken for prevention**. The option finds the error after it happened and is offered as stopping it. Sampling, weekly reconciliation, an audit report. These are correct answers when the scenario asks how anyone finds out, and wrong when it asks how the damage is avoided. Read which of the two was asked.

- **Restructuring mistaken for reassignment**. The option rearranges the work without changing who owns it. Chaining two stages still leaves the judgement in the same place. If a step is wrong because the model holds a fact it cannot hold, no arrangement of steps repairs it.

- **Reversibility misjudged**. The option treats an act that leaves the organisation as though it could be corrected internally, or spends a person on something that could be undone in a minute. Whether an outcome can be withdrawn beats how large it is, every time.

The table below is the compressed form. Cues, not cases: match on what the scenario is doing, not on what industry it is set in.

When the scenario says	The answer is usually	The trap is usually
An accuracy or success rate, with no error breakdown	Ask what the errors are and who absorbs them; the design follows from the answer	Any option that treats the percentage as sufficient to choose a design
The same input must always produce the same result	Move that decision into a deterministic component beside the model	Prompt wording, lower temperature, or repeated sampling, all offered as making output repeat
A fact belongs to the company: pricing, entitlement, a policy exception, a balance	Hand the fact to the model from the system that owns it	Any option that has the model recall it, however well instructed
Free text a person wrote, needing interpretation	The model owns that step	Routing it to a rule or a person out of caution, which is the same misassignment in the other direction

When the scenario says	The answer is usually	The trap is usually
An action that leaves the organisation and cannot be recalled	A person decides, before it goes	Volume-based or amount-based routing, which sends the reversible large thing to review and lets the irreversible small one through
Nobody has noticed a problem yet, and the design is asked to be safe	Build the loop that makes wrongness visible to a named person	Observability, dashboards or logging offered as though a place to look were the same as somebody looking
Several options are each individually sensible and the format wants more than one	Take only the ones that change the outcome, and say what each one changes	The next-most-reasonable good practice, which is the standard fourth option in a set of three

One habit is worth more than the whole table. When two options both look right, find the assumption each one needs in order to be right, then check which assumption the scenario actually supports. The wrong one almost always needs something the scenario never said.

What to Remember

- An **accuracy percentage** is not a design input until you have **counted the errors and attributed** them. Convert every percentage into a number of cases, then ask which cases.
- **Three owners**: **Claude for judgement over ambiguity**, **deterministic systems for facts and calculation**, **people for accountable decisions**. Most design defects you will be shown are a **fact held by the wrong owner**.

- **Claude fails plausibly**: a wrong answer is well formed, raises no error, and looks in every log exactly like a right one. **Deterministic systems fail loudly, people fail slowly**, and the monitoring you already own was built for those two.
- **Non-determinism is a given, even at temperature 0**: identical input is not guaranteed to return identical output. “The same case must always get the same answer” is therefore a **requirement on a deterministic component**, not on the model, and that component belongs in the drawing.
- Place the **delegation boundary**, the line between work handed to the model and work kept under deterministic or human control, with three questions: what the **wrongness costs while nobody notices** it, whether the **outcome can be undone**, and whose **name** is on it. **Stakes route work, not confidence**.
- **Four stages: input, processing, output, feedback**. Most designs put up for review carry the first three. A design with **no feedback loop**, meaning no signal that reaches a named person when the system is wrong, is **not an end-to-end architecture**, whatever the arrows claim.